

Phase 2: RAG Application

POC-07 — Inventory Management & Procurement System

Phase Weight: 20% | **Duration:** 5 working days | **Test Cases:** 20

1. Phase Overview

Build a RAG system for answering questions about inventory policies, procurement procedures, reorder rules, supplier management guidelines, and stock management best practices using the inventory operations manual.

2. Technology Setup

```
pip install langchain langchain-google-genai langchain-community chromadb streamlit
```

Additional .env:

```
GOOGLE_API_KEY=your-gemini-api-key  
LANGCHAIN_API_KEY=your-langsmith-key  
LANGCHAIN_TRACING_V2=true  
LANGCHAIN_PROJECT=AI-Readiness-POC-07-P2
```

3. Knowledge Base — inventory_manual.md

Create `rag/inventory_manual.md`:

```
# Inventory Management & Procurement Operations Manual  
## POC-07 – Retail Inventory Reference Guide  
  
### Section 1: Introduction to Inventory Management
```

```
Inventory management is the process of ordering, storing, and using a company's  
inventory – raw materials, components, and finished products. Effective inventory  
management ensures products are available when customers need them while  
minimizing carrying costs and avoiding overstock situations.
```

```
This system manages four main activities: product catalog management, stock level  
tracking, purchase order management, and stock movement recording. All inventory  
changes are recorded as movements for full auditability.
```

Section 2: Product Catalog and SKU System

Every product has a unique SKU (Stock Keeping Unit) in the format SKU-{CATEGORY_PREFIX}-{NNNN}.

Category prefixes:

- GRO: Grocery products
- ELC: Electronics
- CLO: Clothing and apparel
- HHD: Household items
- PRC: Personal care products

Examples: SKU-GRO-0042 (Grocery item #42), SKU-ELC-0015 (Electronics item #15).

Each product has: *unit_price* (selling price in ₹), *cost_price* (purchase cost in ₹), *unit_of_measure* (pieces/kg/litre/box), *reorder_point* (trigger level for replenishment), and *reorder_quantity* (standard order size).

Section 3: Stock Levels and Reorder Points

Every product has a *StockLevel* record tracking:

- *quantity_on_hand*: physical inventory count
- *quantity_reserved*: allocated to pending customer orders
- *quantity_available*: *on_hand* minus reserved (this is the working stock)

The *reorder_point* is the stock level at which a replenishment order should be initiated. Setting it correctly requires knowing: $\text{average daily demand} \times \text{supplier lead time} + \text{safety stock}$.

Example: A product selling 10 units per day with a 5-day supplier lead time and 2 days safety stock needs a reorder point of $(10 \times 5) + (10 \times 2) = 70$ units.

Section 4: Stock Alert System

The system automatically generates alerts based on stock conditions:

****Low Stock Alert:**** Triggered when $\text{quantity_available} \leq \text{reorder_point}$. This is a warning – action should be taken within 24-48 hours to raise a purchase order.

****Out of Stock Alert:**** Triggered when $\text{quantity_available} = 0$. This is critical – immediate action required. Lost sales occur with every customer request while stock is zero.

****Reorder Suggested Alert:**** The system may also suggest reorder quantities based on historical consumption patterns.

All alerts can be resolved once a purchase order is submitted. Resolved alerts remain in the audit log.

Section 5: Purchase Order (PO) Process

Purchase Orders are the formal mechanism for ordering from suppliers. The PO lifecycle:

1. **Draft:** Created by procurement officer, items added
2. **Submitted:** Sent to supplier (email or system notification)
3. **Acknowledged:** Supplier confirms receipt and delivery date
4. **Received:** Goods arrive, staff marks as received, stock updated automatically
5. **Cancelled:** PO cancelled before receipt

PO number format: PO-{YEAR}-{NNNN}, e.g., PO-2026-0042.

When a PO is marked as received, the system automatically:

- Creates StockMovement(receipt) records for each line item
- Updates quantity_on_hand for each product
- Resolves any low_stock/out_of_stock alerts for received products

Section 6: Supplier Management

Suppliers have the following key attributes:

- supplier_code: unique identifier (SUP-0001 format)
- lead_time_days: how many days from order to delivery
- payment_terms_days: net payment window (e.g., Net 30 = payment within 30 days)
- is_active: only active suppliers can receive new POs

Supplier selection criteria: price competitiveness, Lead time, reliability, and payment terms. Always verify a supplier is active before raising a PO.

Section 7: Stock Movement Recording

Every inventory change must be recorded as a StockMovement with a movement_type:

- **receipt:** Goods received from a PO (positive quantity)
- **sale:** Goods sold to a customer (negative quantity – use negative values)
- **adjustment:** Manual correction after physical count discrepancy
- **transfer:** Moved between warehouses
- **return:** Customer return or supplier return

The reference_number should link to the source document (PO number, sale order number, etc.). All movements are timestamped and attributed to a staff member.

Section 8: Inventory Valuation

Inventory is valued at cost price using the FIFO (First In, First Out) method. Total stock value = SUM(product.cost_price × stock_level.quantity_on_hand) across all products.

The dashboard shows total_stock_value to give management visibility into working capital tied up in inventory. Overstock (inventory far above reorder needs) increases carrying costs.

Section 9: Reorder Quantity Calculation

The Economic Order Quantity (EOQ) formula helps determine optimal reorder quantity:

$$EOQ = \sqrt{2 \times \text{annual_demand} \times \text{ordering_cost} / \text{holding_cost_per_unit}}$$

In practice, most retailers use a simpler rule: order enough to cover lead_time + safety stock period. Example: if a product sells 5 units/day, lead time is 7 days, and safety stock is 14 days: reorder quantity = $(5 \times 7) + (5 \times 14) = 35 + 70 = 105$ units.

Section 10: Procurement Officer Responsibilities

The Procurement Officer (Anita Singh) is responsible for:

1. Monitoring low_stock alerts daily
2. Reviewing supplier catalogs for best prices
3. Raising POs within 24 hours of a low_stock alert
4. Confirming supplier acknowledgement
5. Coordinating goods receipt with warehouse staff

PO approval: POs above ₹50,000 require Store Manager approval before submission.

Section 11: Stock Count and Reconciliation

Physical stock counts should be performed:

- **Cycle count:** Count a subset of products daily/weekly (high-value items weekly)
- **Full count:** Count all products quarterly

When a discrepancy is found: record a StockMovement(adjustment) with the difference. If positive: system had less than physical count (count was understated). If negative: system had more than physical count (shrinkage, theft, or damage).

Section 12: Category Management

Each category has different characteristics:

- **Grocery:** Short shelf life, high velocity, tight reorder management needed
- **Electronics:** High unit cost, lower velocity, longer lead times typical
- **Clothing:** Seasonal demand patterns, size/variant management important
- **Household:** Moderate velocity, predictable demand
- **Personal Care:** High velocity, brand loyalty, competitive market

Section 13: Reporting and Analytics

Key inventory reports:

- **Slow-moving stock:** Products with no movement in 30+ days
- **Stock turn ratio:** Cost of goods sold / average inventory value (higher = better)
- **Fill rate:** Percentage of orders fulfilled without stockout
- **Days on hand:** quantity_on_hand / average daily sales

Store Manager reviews these weekly. Raj Patel (Inventory Analyst) prepares monthly trend reports.

Section 14: System Integration

The inventory system integrates with:

- Point of Sale (POS): sales movements recorded automatically
- Supplier portal: PO submission and acknowledgement

- Finance: PO amounts flow to accounts payable

When integration is not available, manual StockMovement records must be entered.

Section 15: Troubleshooting

****Problem: Stock Level shows negative****

Cause: Sale recorded before receipt, or data entry error.

Fix: Record a positive StockMovement(adjustment) to correct. Investigate the root cause.

****Problem: Duplicate Low_stock alerts****

Cause: Multiple triggers before first alert was resolved.

Fix: The system should check for existing unresolved alert before creating a new one.

****Problem: PO received but stock not updated****

Cause: PATCH /orders/{id}/receive not called, or failed silently.

Fix: Check API logs, verify PO status is "received", manually trigger receipt if needed.

****Problem: SKU not found by supplier****

Cause: Product registered in system but not in supplier catalog.

Fix: Add product to supplier's catalog, verify supplier_id on product record.

4. RAG Pipeline (rag/rag_chain.py)

```
from langchain_google_genai import ChatGoogleGenerativeAI,
GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
from langsmith import traceable

INVENTORY_RAG_PROMPT = PromptTemplate(
    input_variables=["context", "question"],
    template="""You are an inventory management expert assistant for a retail
operations system (POC-07).
Answer questions about inventory policies, procurement procedures, stock
management, and supplier guidelines
using only the provided context.

Context:
{context}

Question: {question}

If the information is not available, say: "I don't have that information in the
inventory manual."
Provide specific rules, formulas, and thresholds where available.
```

```
Answer: ""
)

def build_rag_chain():
    embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
    vectorstore = Chroma(
        collection_name="inventory_manual",
        embedding_function=embeddings,
        persist_directory="./chroma_db"
    )
    llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0.2)
    return RetrievalQA.from_chain_type(
        llm=llm, chain_type="stuff",
        retriever=vectorstore.as_retriever(search_kwargs={"k": 4}),
        return_source_documents=True,
        chain_type_kwargs={"prompt": INVENTORY_RAG_PROMPT}
    )

@traceable(project_name="AI-Readiness-POC-07-P2")
def ask_question(question: str, chain) -> dict:
    result = chain.invoke({"query": question})
    return {"answer": result.get("result", ""), "source_documents":
result.get("source_documents", [])}
```

5. Submission Checklist

- ☐ rag/inventory_manual.md with all 15 sections
- ☐ ChromaDB collection "inventory_manual" with ≥20 chunks
- ☐ SKU format (SKU-GRO-NNNN) retrievable from manual
- ☐ Reorder point rules retrievable
- ☐ PO lifecycle (draft→submitted→received) retrievable
- ☐ Streamlit Q&A interface functional
- ☐ LangSmith project "AI-Readiness-POC-07-P2" traces
- ☐ 20 test cases: ≥14 passing